WRPC updates

This document aims to describe the implementation of updates to wrpc to include IRIG-B, NMEA and configurable clock outputs.

A summary of the updates are:

- provide IRIG-B output
- provide NMEA output ($GPZDA)
- provide configurable clock output (frequency and duty cycle)
- provide option to multiplex clock, IRIG-B and NMEA to single output
- expose leap seconds value (to top level HDL entity)
- expose leap seconds flags (to top level HDL entity)
- add cli tools to wrpc-sw for configuring/monitoring above outputs

These changes are implemented within a single top level wrapper called wr_timecodes, which can be instantiated in the wr_core design.

Building HDL

To get the sources and build for the spec design run the following:

```
git clone https://ohwr.org/project/wr-cores.git
cd wr-cores
git checkout hl-wrpc-v5.0
git submodule init
git submodule update

cd top/spec_ref_design
hdlmake
make
```

Building wrpc-sw

Enabling the additional outputs in wrpc-sw requires some manual configuration (they're not added to a defconfig). To get the sources and build for the spec design run the following:

```
git clone https://ohwr.org/project/wrpc-sw.git
cd wrpc-sw
git checkout hl-wrpc-v5-wrs-v3-urv
git submodule init
git submodule update

make spec_defconfig
make menuconfig
<enable auxiliary timing outputs/commands as needed, see CLI Tools section>
make
```

Implementation

A simplified block diagram of the HDL and software architecture is shown in Figure 1.

The basic operation is that of a simple buffer system. wr_timecodes has two register blocks, next_utc and current_utc. The pps_gen module was updated to expose a pps_pre trigger, set one cycle before the main pps output is generated. This pps_pre is used as a tx_enable to the timecode outputs (IRIG-B, NMEA) such that they start transmitting next_utc on same edge as the main pps out. On this same edge, the values of next_utc are registered into current_utc, thus showing the time that is currently being sent on the additional timecode outputs.

Updating the values in the next_utc register is done by wrpc-sw. wrpc-sw reads the value of the main WR TAI counter in pps_gen. On a change of value i.e new second, it writes the value of the next second to the next_utc register. This time is then output on the timecode outputs (IRIG-B/NMEA) at the start of the next second

It was implemented this way for efficiency (we need to convert a TAI value into single UTC fields, which is already being done in sw), and flexibility (e.g. we can update values on leap second events, add/remove fields). As timecode output frames are sent every 1s, there is plenty of time to do the conversion in software.

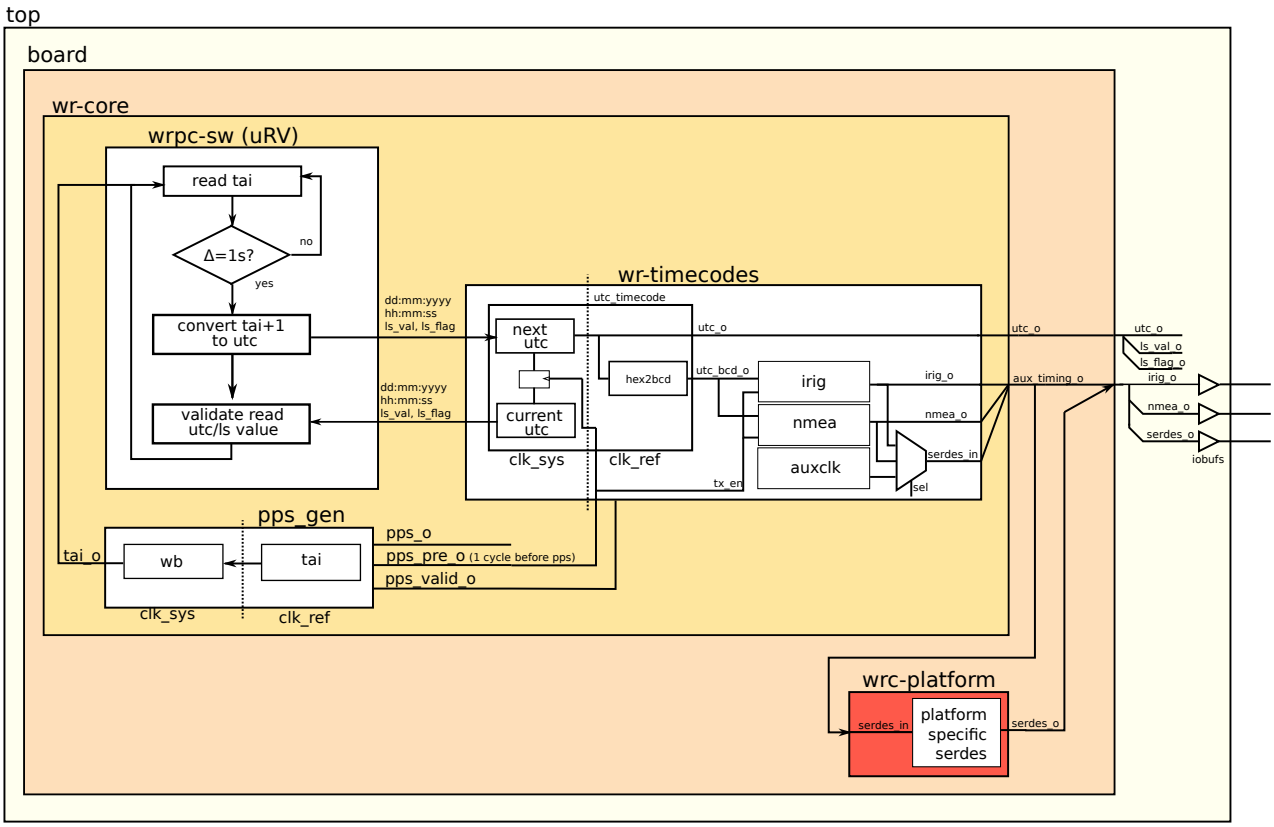

Figure 1: block diagram

All timing outputs are in clk_ref domain, therefore some CDCs are required for the software /register interface. clk_ref is a different frequency depending on the device/phy (125MHz or 62.5MHz).

Auxiliary Clock

The auxiliary clock generator uses the same method as on the wr switch – generate a data word of width N at some frequency f, and serialise this at a rate of N*f. Each bit in the word then corresponds to an output period of 1/(N*f). From this a signal of essentially any frequency and pulse width can be generated.

The serialisation is done using a serdes primitive of the target FPGA. This requires two platform specific primitives – a PLL to generate the serdes clock, and the serdes itself. These primitives are instantiated in the corresponding wrc_platform module. This means there is a dependency on connecting this platform file correctly to use the wr_timecodes module. g_with_serdes must be set in wrc_platform. This option was preferred over instantiating the FPGA primitives directly in wr_timecodes as it is more generic and inline with the existing wrpc structure.

In wr_timecodes, the auxclk module was updated to run from the clk_ref domain. This requires scaling of the word width and serdes PLL settings depending on the clk_ref frequency/device being used. This is set by generics passed to the module: g_ref_clk_rate, g_serdes_data_width. See instantiation in wr_core.vhd which uses existing functions and generics to set these parameters.

Multiplexer

As the original cute-wr reference multiplexed multiple timing outputs to a single port, wr_timecodes also allows for this feature. It is not possible to mux after the serdes (tools will fail p&r), therefore the mux is inserted before the serdes, within the wr_timecodes module. This mux is controlled from wrpc-sw, and can be set via .config and/or cli interface. Cute-wr routes all of its timing outputs through serdes primitives, however due to the data rates of NMEA/IRIG-B it was decided that this is not necessary for wrpc. Further, its only possible to mux on one output and there is only one serdes instantiated. This was done as a tradeoff between keeping the design generic and adding multiplexing functionality. The user can always add more primitives/muxes if they need.

Records

To simplify the design, all of these timing outputs are broken out on two new records: t_utc_out and t_aux_timing_out, defined in wr_timecode_pkg.vhd. t_utc_out contains all individual utc fields (year, month, day etc) for user applications. t_aux_timing_out contains the IRIG-B, NMEA and mux’d serdes input (auxiliary clock/IRIG-B/NMEA) and output.

This package also defines a t_wr_timcode_config, which is simply a boolean array used to enable each of the IRIG-B, NMEA and Auxclk outputs to be synthesised. This is is passed to wr_timecodes via the g_timecode_config generic.

Therefore, to use the additional timing outputs in a design, simply define your configuration with a t_wr_timecode_config and pass to the top level g_aux_timing_config generic of wr_core. Also

enable g_with_serdes in wrc_platform (as stated before) and assign the serdes_i/serdes_o ports. See board/spec/xwrc_board_spec.vhd in wr-cores for an example.

NMEA Invert

Internally, the NMEA generator uses an existing UART core to serialise/format the data to be transmitted. However NMEA uses a serial RS232 interface, therefore there needs to be some conversion between UART and RS232. Normally, this is done using an external RS232 line driver on the target PCB, e.g on WRSv3 MAX3232CDBR is used. This features an inverter and charge pump to convert UART signaling to RS232 levels. During development, most tests were done using a SPEC with DIO FMC board, which has no such line driver. Therefore an option to invert the NMEA output to meet RS232 format was added. The user can enable/disable inverting the NMEA output via wrpc-sw .config or cli command, depending on their implementation. Note that this only inverts the data, the output level is determined by hardware.

Known Issues

Currently, the auxclk generator does not meet timing on Spartan 6 platform. This is due to the loop generating the clock data word in xwr_auxclk_gen.vhd. This method needs to be updated in general, or simplified for this specific platform to resolve this issue. Also, depending on number of users, support for this platform could possibly be dropped.

Leap second value and flags are not updated on a leap second event. Currently, the leap second value is set to that defined in .config. To update this should “just” require a software only change.

---

wrpc-sw config options

The following config options have been added to wrpc-sw:

CONFIG_AUX_TIMING_EN
: enable auxiliary timing outputs

CONFIG_AUXCLK_EN
: enable auxiliary/configurable clock output

CONFIG_AUXCLK_FREQ
: set frequency of auxiliary clock output (Hz)

CONFIG_AUXCLK_DUTY
: set duty cycle of auxiliary clock output (%)

CONFIG_NMEA_EN
: enable NMEA ouput

CONFIG_NMEA_Bxxxx
: set NMEA baud rate, from list of valid bauds

CONFIG_NMEA_INVERT
invert the NMEA output

CONFIG_IRIG_EN
enable IRIG-B output

CONFIG_AUXTMG_SEL_CLK/IRIG/NMEA
selection for serdes input mux.

Commands:

CONFIG_CMD_AUXCLK
enable “auxclk” command

CONFIG_CMD_NMEA
enable “nmea” command

CONFIG_CMD_AUXTMG
enable “auxtmg” command

CLI Tools

The following tools have been added

nmea

usage: nmea
status : readback status of nmea output
baud : set baudrate of nmea output (no argument shows current baud rate)
invert : invert nmea output

auxclk

usage: auxclk
set <frequency (Hz)> <duty (%)> : set auxclk output to <frequency> and <duty>
get : returns current settings

auxtmg

usage: auxtmg
status : read status of auxiliary timing outputs
sel : select for multiplexed output (clk/nmea/irig)

Potentially these could be merged into a single tool if users require.